

UNIVERSITÀ DEGLI STUDI DI SALERNO



DIPARTIMENTO DI INFORMATICA

CORSO DI

RETI SOCIALI

Progetto: Majority dynamics in online networks

<https://github.com/valemosca00/Majority-cascade.git>

Valerio Mosca

mat. 0522501878

Anno Accademico 2025-2026

Indice

1	Definizione del problema, algoritmi e metodologia sperimentale	1
1.1	Selezione del Grafo	2
1.2	Algoritmo 1 - Cost-Seeds Greedy	5
1.3	Algoritmo 2 - WTSS (Weighted Target Set Selection)	6
1.4	Algoritmo 3 - My-Seeds	8
1.5	Struttura degli esperimenti	9
1.6	Ottimizzazioni implementative	10
2	Esperimento 1 - Variazione del budget	12
2.1	Funzione di costo c_1 (random)	12
2.2	Funzione di costo c_2 (grado)	14
3	Esperimento 2 - Robustezza alla rimozione di archi	17
3.1	Effetti strutturali della rimozione di archi	19
3.2	Funzione di costo c_1 (random)	19
3.3	Funzione di costo c_2 (grado)	21
4	Esperimento 3 - Robustezza alla rimozione di vertici	23
4.1	Effetti strutturali della rimozione di vertici	25
4.2	Funzione di costo c_1 (random)	26
4.3	Funzione di costo c_2 (grado)	28
5	Analisi comparativa e conclusioni	29
5.1	Confronto sull'effetto del budget e della funzione di costo	29
5.2	Confronto sulla robustezza alla rimozione di archi	30
5.3	Confronto sulla robustezza alla rimozione di vertici	31
5.4	Sintesi e considerazioni conclusive	32

1 Definizione del problema, algoritmi e metodologia sperimentale

L'obiettivo di questo progetto è la gestione della Majority dynamical process di Influence Diffusion in online networks.

Il problema: Majority Cascade

Sia $G = (V, E)$ una rete sociale. Alcuni nodi vengono "attivati" inizialmente (seed set S). Un nodo non attivato si attiva quando **almeno metà dei suoi vicini** è già attivata. Il processo si ripete finché non si stabilizza (nessun nuovo nodo si attiva).

Formalmente:

- $Inf[S, 0] = S$
- $Inf[S, r] = Inf[S, r - 1] \cup \{v : |N(v) \cap Inf[S, r - 1]| \geq d(v)/2\}$
- Il processo si ferma quando $Inf[S, t] = Inf[S, t + 1]$; il risultato finale è $Inf[S]$

- Ogni nodo v ha un grado $d(v)$ (numero di vicini).
- Un nodo **non attivato** si attiva a un certo step r se **almeno la metà dei suoi vicini è già attiva**: $|N(v) \cap Inf[S, r - 1]| \geq d(v)/2$.
- Una volta attivato, un nodo **resta attivo per sempre** (non c'è "disattivazione").
- Il processo è quindi **monotono crescente**: $Inf[S, 0] \subseteq Inf[S, 1] \subseteq \dots$, e siccome V è finito, prima o poi si stabilizza (arriva a un punto fisso).

È un modello di "contagio sociale": un'idea/comportamento si diffonde se la maggioranza dei tuoi amici l'ha già adottata.

La domanda fondamentale che ci si pone è:

esiste un seed set $S \subseteq V$ con $|S| \leq k$ tale che il processo di majority cascade porti all'attivazione della maggioranza o di **tutto** V , cioè $Inf[S] = V$?

Variante: Majority cascade in cost networks

Ogni nodo u ha un costo $c(u)$ per essere incluso nel seed set (attivarlo "artificialmente" costa).

Dato un budget k , si vuole scegliere il seed set S (con $c(S) \leq k$) che **massimizza** $|Inf[S]|$, cioè il numero totale di nodi che finiscono per attivarsi.

Questo problema è noto per essere NP-hard e difficilmente approssimabile (anche nel caso semplificato con costi unitari $c(u) = 1$). Di conseguenza, la risoluzione pratica richiede l'impiego di **euristiche** per l'individuazione di un seed set massimale S tale che $c(S) \leq k$.

Da specifiche progettuali vengono presi in considerazione due approcci standard :

1. **Algorithm 1 - Cost-Seeds-Greedy:** aggiunge iterativamente il nodo u che massimizza il rapporto "guadagno marginale / costo" ($\Delta_v f_i(S_d) / c(u)$), usando una delle tre funzioni f_1, f_2, f_3 (variano su come "pesano" il contributo di ogni vicino attivato rispetto alla soglia di maggioranza del nodo).
2. **Algorithm 2 - WTSS (Weighted Target Set Selection):** un algoritmo più elaborato che elimina i nodi dal grafo uno alla volta, decidendo caso per caso se un nodo verrà attivato "gratis" dai suoi vicini, se va aggiunto al seed set, o se può ancora aspettare, aggiornando dinamicamente le soglie residue.

Ai quali viene affiancata una terza euristica, definita ed ideata individualmente:

3. **Algorithm 3 - My-Seeds:** un algoritmo greedy basato sulla *k-core decomposition* della rete, che seleziona iterativamente i nodi con il miglior rapporto tra coreness residua e costo, applicando una penalità incrementale ai vicini diretti dei nodi già selezionati per attenuare l'effetto di sovrapposizione tra vicinati densamente connessi.

L'efficacia del seed set S individuato dai tre algoritmi verrà valutata non solo in contesti statici sul grafo originale G , ma anche analizzando la robustezza della propagazione a seguito di modifiche dinamiche della rete, quali la rimozione casuale di archi o vertici.

Nelle sezioni successive verranno descritte in dettaglio ciascuna euristica, la struttura della rete scelta e l'analisi sperimentale dei risultati.

1.1 Selezione del Grafo

Per la valutazione sperimentale è stata scelta la rete *ca-AstroPh*, disponibile nel repository Snap¹. Si tratta di un grafo non orientato che rappresenta una rete di collaborazione scientifica: ciascun nodo corrisponde a un autore di articoli nell'area *Astro Physics* su arXiv, mentre un arco tra due nodi indica che i due autori hanno co-firmato almeno un articolo.

La rete è caratterizzata dalle seguenti proprietà strutturali:

¹<https://snap.stanford.edu/data/ca-AstroPh.html>

Tabella 1: Proprietà strutturali della rete ca-AstroPh

Metrica	Valore
Numero di nodi	18772
Numero di archi	198110
Nodi nella componente connessa più grande	17903 (95.4%)
Archi nella componente connessa più grande	197031 (99.5%)
Numero di triangoli	1 351 441
Frazione di triangoli chiusi	0.1345
Grado medio	21.11
Grado massimo	504
Diametro (shortest path più lungo)	14
Diametro effettivo (90-percentile)	5
Coefficiente di clustering medio	0.6306

La scelta di questo dataset è motivata dal fatto che rappresenta una rete reale (co-authorship network), quindi di rilevanza pratica per applicazioni concrete di influence diffusion, e dalle sue proprietà topologiche distintive. La presenza di autori che collaborano con un numero elevato di coautori (hub), di sottostrutture comunitarie legate a gruppi di ricerca e collaborazioni ricorrenti, e di una **distribuzione dei gradi fortemente eterogenea**, tipica delle reti reali, la rendono un banco di prova adatto per valutare l'efficacia degli algoritmi di seed set selection e per osservare dinamiche di diffusione in scenari complessi e realistici.

Analizzando la rete più nel dettaglio si osserva:

- **Diametro effettivo basso:** un diametro effettivo (90-percentile) pari a 5, a fronte di un diametro esatto di 14, indica che la stragrande maggioranza delle coppie di nodi è raggiungibile in pochissimi passi, mentre il valore massimo di 14 è determinato da un numero ristretto di percorsi periferici, tipico delle reti reali con struttura small-world.
- **Clustering elevato:** un coefficiente di clustering medio di 0.6306, sensibilmente più alto di quanto atteso in un grafo casuale di pari dimensione e densità, indica una forte tendenza dei coautori di un dato ricercatore ad essere anch'essi coautori tra loro — coerente con la natura stessa delle collaborazioni scientifiche, che tendono a formarsi all'interno di gruppi di ricerca stabili.
- **Componente connessa dominante:** il 95.4% dei nodi e il 99.5% degli archi ricadono in un'unica componente connessa, il che garantisce che la quasi totalità della rete possa effettivamente partecipare a un processo di diffusione, rendendo gli esperimenti rappresentativi dell'intera rete e non solo di un suo frammento isolato.

Queste caratteristiche (diametro effettivo basso, clustering elevato, componente gigante dominante) sono proprie delle reti *small-world*, secondo il modello di Watts-Strogatz. Ciò rende la rete particolarmente idonea allo studio del *majority cascade*: da un lato l'alto clustering implica vicinati fortemente sovrapposti, favorendo l'attivazione per maggioranza quando un piccolo gruppo di coautori collegati viene incluso nel seed set; dall'altro il diametro effettivo contenuto permette a un'informazione (o a un comportamento) di raggiungere potenzialmente l'intera rete in un numero ridotto di iterazioni della cascata, rendendo osservabili in tempi computazionali ragionevoli gli effetti delle diverse strategie di selezione del seed set.

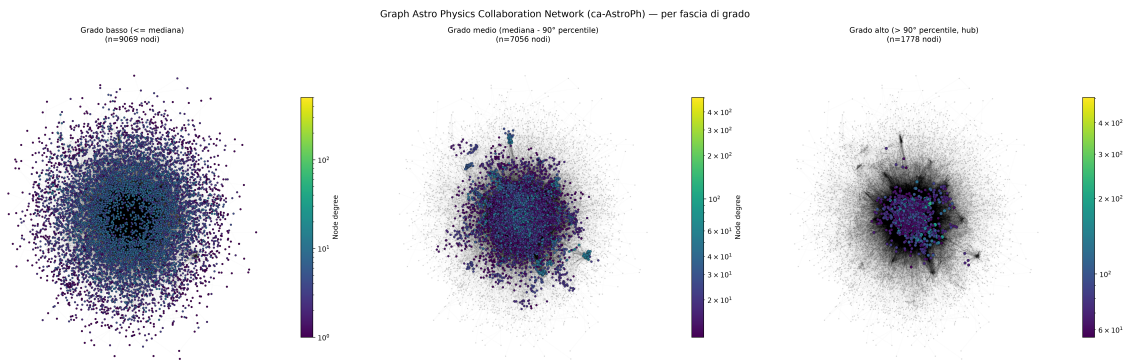


Figura 1: Visualizzazione della rete ca-AstroPh suddivisa per fasce di grado dei nodi. I tre pannelli evidenziano rispettivamente i nodi con grado basso (fino alla mediana), medio (tra la mediana e il 90° percentile) e alto (oltre il 90° percentile, corrispondenti agli hub della rete), mantenendo lo stesso layout spaziale per garantire la confrontabilità tra i pannelli. In ciascun pannello, i nodi non appartenenti alla fascia evidenziata sono mostrati in grigio chiaro come riferimento di contesto, mentre i nodi della fascia sono colorati secondo il proprio grado (scala logaritmica) e dimensionati proporzionalmente ad esso.

Per trasparenza metodologica, per garantire un'analisi rigorosa e la corretta esecuzione degli algoritmi previsti, durante la fase di preprocessing del grafo sono stati identificati e rimossi 60 *self-loop*. Tale scelta risponde a precisi vincoli tecnici e strutturali legati all'analisi di rete:

- **Compatibilità con le euristiche:** La terza euristica adottata fa uso della funzione `nx.core_number()` della libreria *NetworkX*, la quale richiede espressamente che il grafo sia privo di *self-loop* per il corretto calcolo del *k*-core decomposition.
- **Distorsione delle metriche di grado:** Nei grafi gestiti tramite *NetworkX*, un *self-loop* incrementa di 2 il valore del grado totale del nodo interessato, falsandone le reali caratteristiche di connettività e alterando la valutazione relativa della centralità dei nodi all'interno della rete.

La loro eliminazione preventiva assicura dunque sia la stabilità computazionale della pipeline sia la coerenza interpretativa delle metriche topologiche estratte.

1.2 Algoritmo 1 - Cost-Seeds Greedy

L'algoritmo COST-SEEDS-GREEDY costruisce un seed set S in maniera incrementale, selezionando ad ogni iterazione il nodo che offre il miglior rapporto tra guadagno marginale e costo, secondo una funzione surrogata f_i ($i = 1, 2, 3$) che approssima l'effetto della cascata senza doverla simulare interamente ad ogni passo.

Algorithm 1 Cost-Seeds-Greedy(G, k, c, f_i)

```

1:  $S_p \leftarrow \emptyset; S_d \leftarrow \emptyset$ 
2: repeat
3:   seleziona  $u = \arg \max_{v \in V - S_d} \frac{\Delta_v f_i(S_d)}{c(v)}$ 
4:    $S_p \leftarrow S_d$ 
5:    $S_d \leftarrow S_p \cup \{u\}$ 
6: until  $c(S_d) > k$ 
7: return  $S_p$ 

```

dove $\Delta_v f_i(S_d) = f_i(S_d \cup \{v\}) - f_i(S_d)$ rappresenta il guadagno marginale ottenuto aggiungendo il nodo v all'insieme corrente S_d . L'algoritmo termina quando il costo di S_d supera il budget k , restituendo l'ultimo seed set ancora valido, S_p .

Le tre funzioni considerate sono:

$$f_1(S) = \sum_{v \in V} \min \left\{ |N(v) \cap S|, \left\lceil \frac{d(v)}{2} \right\rceil \right\} \quad (1)$$

$$f_2(S) = \sum_{v \in V} \sum_{i=1}^{|N(v) \cap S|} \max \left\{ \left\lceil \frac{d(v)}{2} \right\rceil - i + 1, 0 \right\} \quad (2)$$

$$f_3(S) = \sum_{v \in V} \sum_{i=1}^{|N(v) \cap S|} \max \left\{ \frac{\left\lceil \frac{d(v)}{2} \right\rceil - i + 1}{d(v) - i + 1}, 0 \right\} \quad (3)$$

- f_1 conta, per ciascun nodo v , quanti vicini di v sono in S , arrestando il conteggio non appena viene raggiunto il numero $\lceil d(v)/2 \rceil$ necessario per attivare v : eventuali vicini aggiuntivi oltre questo punto non incrementano ulteriormente il suo contributo. In questo modo f_1 scoraggia l'algoritmo dal concentrare risorse su nodi già "sufficientemente convinti", a vantaggio di nodi non ancora saturati.
- f_2 pesa in modo decrescente il contributo dei vicini in S : il primo vicino contribuisce con il valore massimo della soglia, i successivi con un peso via via minore. Spargere l'influenza su tanti nodi (dando a ciascuno solo un po' di spinta) è desiderabile dato che quei nodi parzialmente convinti potrebbero comunque completarsi a vicenda più avanti nel processo di diffusione naturale, senza bisogno che vengano completati tutti direttamente dal seed set.

- f_3 è analoga a f_2 ma normalizza il contributo rispetto al grado residuo del nodo, tenendo conto della maggiore difficoltà di attivare nodi ad alto grado. Avvicinare di un passo un nodo che ha soglia bassa non ha lo stesso impatto che farlo per uno con soglia alta, motivo per cui lo stesso schema di pesi decrescenti di f_2 viene diviso per il grado residuo del nodo.

Poiché le tre funzioni producono in generale seed set diversi a parità di k e c , COST-SEEDS-GREEDY con f_1 , f_2 e f_3 viene trattato, ai fini del confronto sperimentale, come tre algoritmi distinti.

1.3 Algoritmo 2 - WTSS (Weighted Target Set Selection)

A differenza di COST-SEEDS-GREEDY, l'algoritmo WTSS non costruisce il seed set aggiungendo nodi dall'esterno, ma opera per eliminazione progressiva dei nodi dal grafo, decidendo per ciascuno se sarà attivato naturalmente dai suoi vicini oppure se deve essere incluso forzatamente in S .

Algorithm 2 WTSS(G)

```
1: Input: un grafo  $G = (V, E)$  con soglie  $t(v)$  e costi  $c(v)$ , per  $v \in V$ .
2: Output: un target set  $S$  per  $G$ .
3:  $S \leftarrow \emptyset$ ;  $U \leftarrow V$ 
4: for ogni  $v \in V$  do
5:    $\delta(v) \leftarrow d_G(v)$ ;  $k(v) \leftarrow t(v)$ ;  $N(v) \leftarrow \Gamma_G(v)$ 
6: end for
7: while  $U \neq \emptyset$  do
8:   if esiste  $v \in U$  t.c.  $k(v) = 0$  then ▷ Caso 1:  $v$  è già attivato dai vicini fuori  $U$ 
9:     for ogni  $u \in N(v)$  do
10:       $k(u) \leftarrow \max\{0, k(u) - 1\}$ 
11:    end for
12:   else if esiste  $v \in U$  t.c.  $\delta(v) < k(v)$  then ▷ Caso 2:  $v$  non può attivarsi naturalmente
13:      $S \leftarrow S \cup \{v\}$ 
14:     for ogni  $u \in N(v)$  do
15:       $k(u) \leftarrow k(u) - 1$ 
16:    end for
17:   else ▷ Caso 3: nessuna decisione ancora necessaria per  $v$ 
18:      $v \leftarrow \arg \max_{u \in U} \left\{ \frac{c(u) k(u)}{\delta(u)(\delta(u) + 1)} \right\}$ 
19:   end if
20:   for ogni  $u \in N(v)$  do
21:      $\delta(u) \leftarrow \delta(u) - 1$ ;  $N(u) \leftarrow N(u) - \{v\}$ 
22:   end for
23:    $U \leftarrow U - \{v\}$ 
24: end while
```

L'algoritmo mantiene, per ogni nodo $v \in U$, un grado residuo $\delta(v)$ (numero di vicini ancora in U) e una soglia residua $k(v)$ (numero di vicini attivi ancora necessari per l'attivazione di v).

Ad ogni iterazione si distinguono tre casi:

- se $k(v) = 0$, il nodo v è già garantito attivo grazie ai nodi già processati, e il beneficio viene propagato decrementando la soglia residua dei suoi vicini in U ;
- se $\delta(v) < k(v)$, i vicini rimanenti di v non sono sufficienti a raggiungere la soglia richiesta anche nel caso migliore, per cui v viene aggiunto forzatamente a S ;
- altrimenti, la decisione su v viene rimandata e si seleziona, tramite l'euristica basata su costo, soglia residua e grado, il prossimo nodo da rimuovere dal grafo.

Poiché WTSS non include nativamente un vincolo di budget, per l'applicazione al problema con costo l'algoritmo viene fatto terminare non appena il seed set S costruito raggiunge il massimo costo consentito entro il budget k .

1.4 Algoritmo 3 - My-Seeds

L'algoritmo proposto, denominato MY-SEEDS, si basa sul concetto di *k-core decomposition*: a differenza del grado, che misura semplicemente il numero di vicini di un nodo, la *coreness* di un nodo v misura quanto v sia strutturalmente annidato in un nucleo denso della rete, ovvero quanto i suoi vicini siano a loro volta ben connessi tra loro.

Formalmente:

La *coreness* di v è il valore massimo k tale che v appartenga al k -core del grafo, cioè al più grande sottografo in cui ogni nodo ha grado interno almeno k .

La scelta di questo criterio è **motivata dalle proprietà strutturali della rete ca-AstroPh** utilizzata per gli esperimenti: un coefficiente di clustering medio elevato (0.6306) indica la presenza di numerosi nuclei densamente interconnessi, tipici dei gruppi di ricerca che collaborano ripetutamente tra loro.

In una simile struttura, un nodo con *coreness* alta possiede vicini a loro volta fortemente connessi, **condizione favorevole all'attivazione per maggioranza**: la probabilità che un vicino di un nodo già attivo sia a sua volta supportato da altri vicini attivi risulta più alta rispetto a un nodo con *coreness* bassa ma pari grado, situato ad esempio in una struttura a stella priva di interconnessioni locali.

Un ranking statico basato sulla sola *coreness*, tuttavia, non tiene conto della **sovrapposizione tra i vicinati dei nodi selezionati**: due nodi con *coreness* elevata appartenenti allo stesso nucleo denso rischiano di influenzare in gran parte lo stesso insieme di vicini, rendendo ridondante l'inclusione di entrambi nel seed set.

Per mitigare questo effetto, l'algoritmo applica una penalità permanente e incrementale ai vicini diretti di ciascun nodo appena selezionato, riducendone il punteggio residuo in maniera proporzionale al numero di vicini già inclusi in S , pur senza escluderli definitivamente dalla selezione.

Algorithm 3 My-Seeds(G, k, c)

```
1: calcola  $\text{core}(v)$  per ogni  $v \in V$  ( $k$ -core decomposition, calcolata una sola volta)
2:  $\text{score}(v) \leftarrow \text{core}(v)$  per ogni  $v \in V$ 
3:  $S \leftarrow \emptyset$ ;  $C \leftarrow V$  ▷  $C$ : insieme dei candidati residui
4: while esiste  $v \in C$  tale che  $c(S \cup \{v\}) \leq k$  do
5:    $u = \arg \max_{v \in C} \frac{\text{score}(v)}{c(v)}$ 
6:    $S \leftarrow S \cup \{u\}$ 
7:    $C \leftarrow C - \{u\}$ 
8:   for ogni  $w \in N(u) \cap C$  do
9:      $\text{score}(w) \leftarrow \max\{0, \text{score}(w) - 1\}$ 
10:  end for
11: end while
12: return  $S$ 
```

Il criterio di selezione $\text{score}(v)/c(v)$ privilegia nodi con un **buon rapporto tra centralità strutturale e costo di attivazione**, coerentemente con l'approccio adottato negli altri algoritmi considerati. Rispetto a COST-SEEDS-GREEDY, che ricalcola ad ogni iterazione il guadagno marginale esatto rispetto all'intero seed set corrente, MY-SEEDS adotta un **meccanismo di aggiornamento più leggero**, limitato ai soli vicini diretti del nodo appena selezionato, con un costo computazionale per iterazione pari a $O(\deg(u))$ anziché a una ricomputazione sull'intera rete.

Tale scelta rappresenta un **compromesso tra qualità della soluzione e tempo di esecuzione**: l'algoritmo non raggiunge la stessa capacità di adattamento dinamico di COST-SEEDS-GREEDY nel rilevare sovrapposizioni indirette tra i vicinati (ossia sovrapposizioni non limitate ai soli vicini diretti dell'ultimo nodo selezionato), ma risulta significativamente più efficiente su reti di grandi dimensioni come quella utilizzata in questo progetto.

1.5 Struttura degli esperimenti

Il grafo di riferimento $G = (V, E)$ utilizzato è sempre lo stesso (ca-AstroPh); a partire da esso vengono definite le diverse configurazioni sperimentali variando l'assegnazione di costo ai nodi, l'algoritmo di selezione del seed set, il budget k e, negli esperimenti di robustezza, la struttura della rete attraverso la rimozione progressiva di archi o vertici.

Funzioni di costo. Vengono considerate due funzioni di costo $c : V \rightarrow \mathbb{N}$:

- $c_1(u) =$ valore casuale scelto uniformemente nel range $[1, 21]$ (generato una sola volta con seed fissato (42), per garantire la riproducibilità e la comparabilità tra le diverse configurazioni sperimentali);
- $c_2(u) = \left\lceil \frac{d(u)}{2} \right\rceil$, deterministica in funzione del grado del nodo.

Algoritmi e varianti. Vengono confrontate complessivamente **cinque curve** per ciascun grafico, corrispondenti a:

- COST-SEEDS-GREEDY con funzione surrogata f_1 ;
- COST-SEEDS-GREEDY con funzione surrogata f_2 ;
- COST-SEEDS-GREEDY con funzione surrogata f_3 ;
- WTSS (Weighted Target Set Selection), troncato al budget k ;
- MY-SEEDS, l'euristica proposta.

Le tre varianti di COST-SEEDS-GREEDY sono trattate come algoritmi distinti, in quanto f_1, f_2 ed f_3 producono in generale seed set differenti a parità di k e c .

Esperimento 1 – Variazione del budget k . Per ciascuna funzione di costo $c \in \{c_1, c_2\}$ e per ciascuno dei cinque algoritmi A , si calcola il seed set $S = A(G, k, c)$ al variare di k , fissato rispettivamente allo 0.5%, 1%, 2%, 5%, 10% e 20% del costo totale della rete, e si misura $|Inf[G, S]|$ tramite simulazione della cascata su G .

Si ottengono così 2 grafici (uno per funzione di costo), ciascuno con 5 curve, aventi k in ascissa e $|Inf[G, S]|$ in ordinata.

Esperimento 2 – Robustezza alla rimozione di archi. Per ciascuna funzione di costo c e per ciascun algoritmo A , si fissa il budget rappresentativo $k^* = 10\%$ del costo totale della rete e si riutilizza il seed set $S = A(G, k^*, c)$ già calcolato nell’Esperimento 1, con il relativo valore $|Inf[G, S]|$ come baseline. Vengono considerate rimozioni pari all’1%, 5%, 10% e 20% degli archi originari. Gli insiemi di archi rimossi sono cumulativi, in modo che $R_{1\%} \subset R_{5\%} \subset R_{10\%} \subset R_{20\%}$: ciascun livello rappresenta quindi un aggravamento della stessa perturbazione casuale. Per ogni livello si costruisce G' , si riutilizza lo stesso S (senza ricalcolarlo su G') e si misura $|Inf[G', S]|$. Durante la rimozione viene inoltre evitata la creazione di nodi isolati.

Si ottengono 2 grafici (uno per funzione di costo), ciascuno con 5 curve, aventi la percentuale di archi rimossi in ascissa e $|Inf[G', S]|$ in ordinata, confrontabili con la baseline $|Inf[G, S]|$.

Esperimento 3 – Robustezza alla rimozione di vertici. Analogamente all’Esperimento 2, ma rimuovendo l’1%, 5%, 10% e 20% dei vertici originari, insieme ai relativi archi incidenti, per ottenere il grafo perturbato G'' . Anche in questo caso gli insiemi di vertici rimossi sono cumulativi, in modo che $R_{1\%}^V \subset R_{5\%}^V \subset R_{10\%}^V \subset R_{20\%}^V$. Il seed set di riferimento rimane quello calcolato nell’Esperimento 1 con $k^* = 10\%$; tuttavia, qualora alcuni seed appartengano all’insieme dei vertici rimossi, il seed set effettivamente disponibile diventa $S' = S \setminus R_V$, senza sostituire i seed eliminati. Per ciascun livello di perturbazione si misura quindi $|Inf[G'', S']|$.

Si ottengono altri 2 grafici con la medesima struttura.

Riepilogo. In totale si producono 6 grafici (2 per esperimento), ciascuno contenente 5 curve (una per algoritmo/variante). Il seed set di riferimento utilizzato negli Esperimenti 2 e 3 per un dato algoritmo e una data funzione di costo è il medesimo calcolato nell’Esperimento 1 in corrispondenza di k^* , evitando ricalcoli ridondanti; nell’Esperimento 3 vengono semplicemente esclusi da tale insieme gli eventuali seed rimossi insieme ai vertici perturbati.

1.6 Ottimizzazioni implementative

Nell’implementazione degli algoritmi è stata posta particolare attenzione alla gestione efficiente della selezione dei candidati. Tale aspetto risulta rilevante sulla rete ca-AstroPh,

composta da diverse migliaia di vertici e da quasi duecentomila archi, **poiché una scansione completa dell'insieme dei candidati a ogni iterazione può comportare tempi di esecuzione elevati.**

Per WTSS e My-Seeds sono state quindi utilizzate strutture dati che consentono di recuperare rapidamente il prossimo vertice da processare e di aggiornare soltanto le informazioni effettivamente modificate.

- Nel caso di WTSS, i tre casi previsti dall'algoritmo vengono gestiti mediante strutture distinte. I vertici appartenenti al Caso 1 e al Caso 2 sono memorizzati in code, poiché all'interno di tali categorie non è necessario mantenere un particolare ordinamento. Per il Caso 3 viene invece utilizzata una coda di priorità basata su heap, ordinata secondo il valore

$$\frac{c(v)k(v)}{\delta(v)(\delta(v) + 1)}.$$

Poiché il modulo `heapq` di Python implementa un min-heap, i valori dello score vengono inseriti con segno negativo, ottenendo così il comportamento equivalente a un max-heap.

Durante l'esecuzione di WTSS, la modifica di $\delta(v)$ o $k(v)$ può rendere obsolete alcune entry precedentemente inserite nello heap. La loro rimozione immediata richiederebbe operazioni aggiuntive e una gestione più complessa della struttura. È stata pertanto adottata una strategia di *lazy deletion*: quando lo score di un vertice cambia, viene semplicemente inserita nello heap una nuova entry aggiornata, senza eliminare quella precedente. Al momento dell'estrazione, il valore recuperato viene confrontato con lo score corrente del vertice; se i due valori non corrispondono, l'entry viene riconosciuta come obsoleta e scartata.

- Una strategia analoga è stata utilizzata per My-Seeds. In questo caso la coda di priorità è ordinata secondo il rapporto

$$\frac{\text{score}(v)}{c(v)},$$

dove lo score iniziale corrisponde alla coreness del vertice. Quando un nodo viene aggiunto al seed set, lo score dei suoi vicini ancora candidati viene decrementato per penalizzare la sovrapposizione dei vicinati. Poiché tali aggiornamenti modificano il rapporto utilizzato come priorità, anche in My-Seeds vengono inserite nuove entry nello heap e quelle obsolete vengono eliminate solamente al momento dell'estrazione attraverso una verifica del valore corrente.

L'impiego dello heap evita quindi di ricercare il candidato migliore mediante una nuova scansione completa di tutti i vertici ancora disponibili a ogni iterazione. Gli aggiornamenti sono invece concentrati sui nodi direttamente interessati dalla modifica dello stato corrente, rendendo l'implementazione più adatta all'esecuzione su reti di dimensioni elevate.

- Cost-Seeds-Greedy mantiene invece un'impostazione più diretta.

2 Esperimento 1 - Variazione del budget

Il primo esperimento ha l'obiettivo di analizzare il comportamento degli algoritmi al variare del budget disponibile per la selezione del seed set.

A tal fine, per ciascuna delle due funzioni di costo considerate, il budget k è stato espresso come percentuale del costo complessivo della rete e valutato sui valori 0.5%, 1%, 2%, 5%, 10% e 20%. Questa scelta consente di osservare sia il comportamento degli algoritmi in condizioni di budget fortemente limitato, sia la loro evoluzione all'aumentare delle risorse disponibili, fino a individuare eventuali fenomeni di saturazione della diffusione.

Le due funzioni di costo sono state definite in modo da rappresentare due scenari differenti. La prima, indicata con c_1 , assegna a ciascun nodo un costo intero estratto uniformemente nell'intervallo $[1, 21]$.

La scelta dell'estremo superiore pari a 21 è stata effettuata per rendere l'ordine di grandezza dei costi confrontabile con quello della seconda funzione di costo. Infatti, per una distribuzione uniforme discreta nell'intervallo $[1, 21]$, il valore medio atteso è

$$\frac{1 + 21}{2} = 11,$$

molto vicino al costo medio osservato per la funzione c_2 , pari a circa 11.26.

Tale scelta consente quindi di mantenere comparabili il costo complessivo della rete e, conseguentemente, i budget percentuali utilizzati nei due scenari, senza tuttavia rendere equivalenti le due distribuzioni.

In c_1 i costi sono infatti **indipendenti dalla struttura del grafo** e distribuiti in maniera uniforme, mentre in c_2 il costo di ciascun nodo è determinato dal suo grado, secondo $c_2(v) = \lceil d(v)/2 \rceil$. Quest'ultima distribuzione risulta pertanto maggiormente eterogenea e caratterizzata da costi più elevati in corrispondenza dei nodi ad alto grado.

Per ogni valore di k sono stati confrontati i cinque metodi considerati: Cost-Seeds-Greedy nelle tre varianti associate alle funzioni surrogate f_1 , f_2 e f_3 , WTSS e My-Seeds. La misura principale utilizzata per il confronto è il numero finale di nodi influenzati, $|Inf[G, S]|$, ottenuto applicando il processo di Majority Cascade al seed set prodotto da ciascun algoritmo.

2.1 Funzione di costo c_1 (random)

In questo scenario, per quanto detto in precedenza, un nodo altamente connesso non risulta necessariamente più costoso di un nodo periferico.

La Figura 2 mostra il numero di nodi complessivamente influenzati, $|Inf[G, S]|$, ottenuto dai cinque metodi considerati al crescere del budget k .

La linea orizzontale tratteggiata rappresenta il numero totale di vertici della rete, pari a 17 903, e consente di individuare immediatamente le configurazioni nelle quali la cascata raggiunge una condizione prossima alla saturazione.

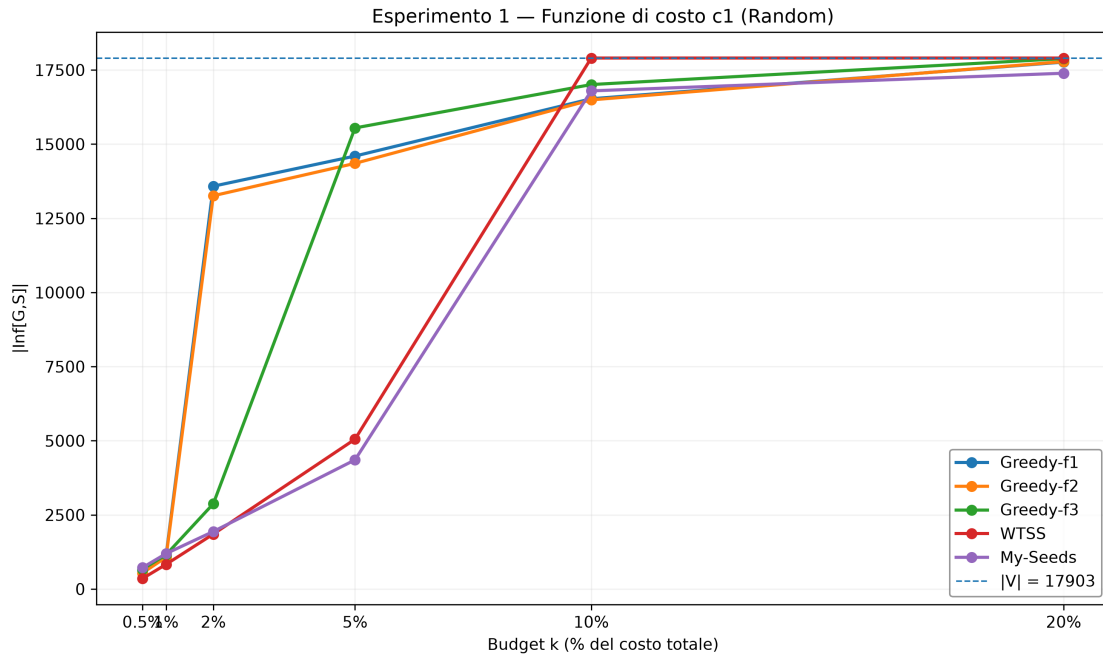


Figura 2: Numero di nodi influenzati al variare del budget per la funzione di costo c_1 .

- Per valori di budget molto ridotti, pari allo 0.5% e all'1% del costo complessivo, tutti gli algoritmi producono cascade relativamente contenute e le differenze tra le diverse strategie risultano ancora limitate. In questa fase il vincolo economico è particolarmente restrittivo e permette di selezionare soltanto un numero ridotto di seed, rendendo difficile innescare una diffusione estesa all'interno della rete.
- Un primo cambiamento significativo si osserva con un budget pari al 2%. Le varianti Greedy associate a f_1 e f_2 mostrano un marcato incremento della diffusione, raggiungendo già oltre 13 000 nodi influenzati. Le restanti strategie presentano invece, a questo livello di budget, una diffusione considerevolmente più limitata. In particolare, Greedy- f_3 , WTSS e My-Seeds rimangono ancora lontani dai livelli raggiunti dalle prime due varianti. Questo comportamento evidenzia come, in condizioni di budget ridotto, la funzione utilizzata per valutare il guadagno marginale dei candidati possa incidere notevolmente sulla capacità di individuare un piccolo insieme di seed sufficientemente efficace da avviare una cascata estesa.
- Con l'aumento del budget al 5%, Greedy- f_3 supera le altre due varianti di Cost-Seeds-Greedy e raggiunge una diffusione superiore a 15 000 nodi. Greedy- f_1 e Greedy- f_2 continuano a produrre risultati elevati, mentre WTSS e My-Seeds rimangono ancora sensibilmente al di sotto. Tale risultato mostra che il comportamento degli algoritmi non è uniforme rispetto all'aumento di k : una strategia meno efficace in presenza di budget molto limitati può diventare maggiormente competitiva quando dispone di risorse sufficienti per selezionare un insieme di seed più ampio.

- Il passaggio al 10% determina un ulteriore cambiamento rilevante. Tutte le varianti Greedy e My-Seeds raggiungono ormai una porzione molto elevata della rete, mentre WTSS arriva praticamente alla saturazione, influenzando tutti i 17 903 vertici. Questo risultato è particolarmente interessante poiché WTSS, che ai budget inferiori aveva mostrato una diffusione più contenuta, presenta una crescita molto pronunciata tra il 5% e il 10%. L'algoritmo sembra quindi necessitare di un budget sufficientemente ampio prima che la struttura del seed set individuato riesca ad attivare una cascata globale.
- Con il budget massimo considerato, pari al 20%, quasi tutte le strategie raggiungono una condizione di saturazione. Greedy- f_1 , Greedy- f_2 , Greedy- f_3 e WTSS influenzano sostanzialmente l'intera rete, mentre My-Seeds rimane leggermente al di sotto del massimo teorico. Di conseguenza, le differenze osservate ai budget inferiori tendono progressivamente a scomparire all'aumentare delle risorse disponibili.

L'andamento complessivo mostra quindi l'esistenza di differenti regimi di funzionamento. Per budget molto ridotti, la scelta dell'algoritmo ha un impatto sostanziale sul risultato e Greedy- f_1 e Greedy- f_2 risultano particolarmente efficaci. A livelli intermedi emergono invece Greedy- f_3 e, successivamente, WTSS, mentre per budget elevati il problema diventa progressivamente meno discriminante, poiché quasi tutti i metodi riescono a generare una cascata prossima alla totalità della rete.

Un ulteriore elemento rilevante riguarda il rendimento marginale dell'aumento del budget. Una volta raggiunto il 10%, gran parte degli algoritmi influenza già una quota molto elevata dei nodi; il successivo raddoppio del budget fino al 20% produce pertanto un miglioramento relativamente contenuto rispetto alla quantità aggiuntiva di risorse impiegate.

Tale comportamento suggerisce la presenza di un fenomeno di saturazione e costituisce una delle motivazioni per cui, **nei successivi esperimenti di robustezza, il 10% è stato scelto come valore rappresentativo k^*** : esso consente di lavorare in una regione nella quale le soluzioni sono già sufficientemente efficaci, evitando al tempo stesso il regime di completa saturazione osservato con il budget massimo.

2.2 Funzione di costo c_2 (grado)

Nel secondo scenario sperimentale è stata utilizzata la funzione di costo c_2 , definita come

$$c_2(v) = \left\lceil \frac{d(v)}{2} \right\rceil.$$

A differenza di c_1 , il costo di un vertice non è quindi indipendente dalla struttura della rete, ma cresce con il suo grado. **I nodi maggiormente connessi**, che potrebbero rappresentare candidati particolarmente interessanti per l'avvio della diffusione, **risultano anche più costosi da inserire nel seed set**.

La distribuzione dei costi è inoltre fortemente eterogenea, in conseguenza della distribuzione dei gradi della rete: la maggior parte dei nodi presenta costi relativamente contenuti, mentre pochi vertici ad alto grado assumono valori significativamente superiori alla media.

La Figura 3 mostra l'andamento del numero di nodi influenzati al variare del budget per i cinque metodi considerati.

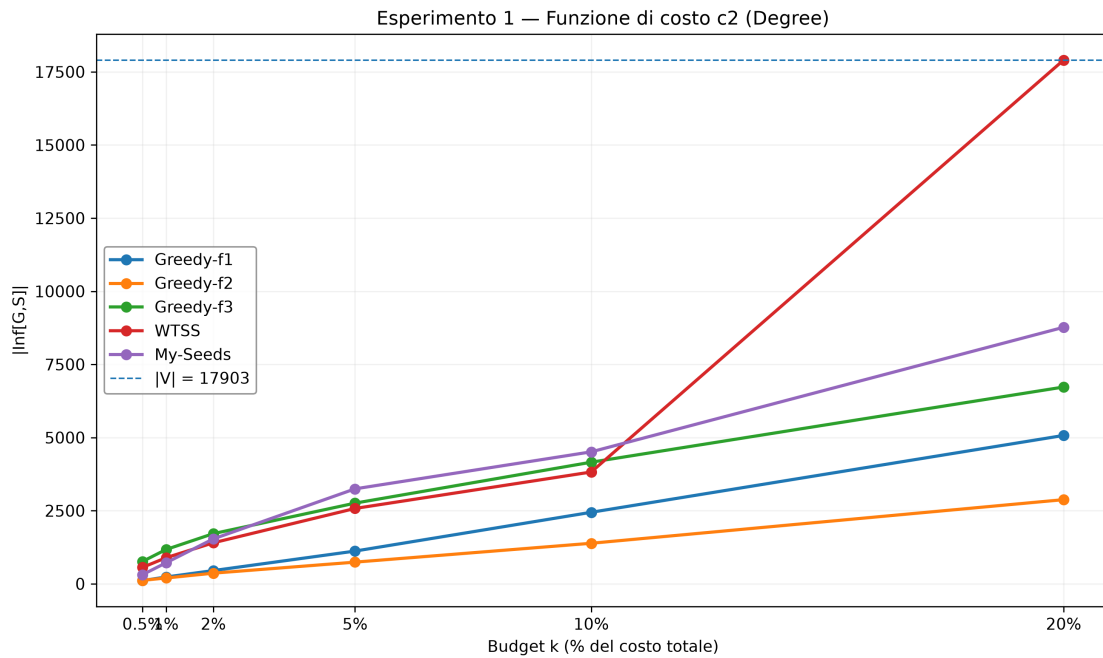


Figura 3: Numero di nodi influenzati al variare del budget per la funzione di costo c_2 .

Rispetto allo scenario basato sulla funzione di costo random, il comportamento cambia in maniera evidente. Per quasi tutti i valori di budget considerati, il numero di nodi influenzati rimane sensibilmente inferiore rispetto a quanto osservato con c_1 .

Questo risultato può essere ricondotto alla natura stessa della funzione di costo: assegnando un costo maggiore ai nodi con grado elevato, l'accesso ai vertici maggiormente connessi diventa più oneroso e **gli algoritmi sono incentivati a selezionare nodi meno costosi, ma non necessariamente altrettanto efficaci nell'innescare una cascata estesa.**

- Per budget molto ridotti, compresi tra lo 0.5% e il 2%, tutte le strategie producono una diffusione limitata. Le differenze tra gli algoritmi sono già visibili, ma nessuno dei metodi riesce a raggiungere una porzione consistente della rete. In questa regione, Greedy- f_3 , WTSS e My-Seeds mostrano generalmente una diffusione superiore rispetto a Greedy- f_1 e Greedy- f_2 , evidenziando una diversa capacità delle funzioni surrogate e delle euristiche considerate di bilanciare il rapporto tra costo dei nodi e potenziale contributo alla cascata.

- Aumentando il budget al 5%, My-Seeds raggiunge il risultato migliore tra i cinque metodi, mentre Greedy- f_3 e WTSS si collocano immediatamente al di sotto. Greedy- f_1 e soprattutto Greedy- f_2 continuano invece a produrre una diffusione più contenuta. **La differenza con lo scenario c_1 è significativa:** a parità di percentuale di budget rispetto al costo totale della rete, non si osserva il rapido avvicinamento alla saturazione che caratterizzava la funzione di costo random.
- Con un budget pari al 10%, la separazione tra gli algoritmi rimane evidente. My-Seeds conserva il valore più elevato di diffusione, seguito da Greedy- f_3 , WTSS, Greedy- f_1 e Greedy- f_2 . Nessuno dei metodi si trova tuttavia vicino alla saturazione dell'intera rete.
- Il comportamento più marcato si osserva con il passaggio dal 10% al 20%. Mentre Greedy- f_1 , Greedy- f_2 , Greedy- f_3 e My-Seeds mostrano una crescita sostanzialmente progressiva, WTSS presenta un incremento molto più pronunciato e, con il budget massimo, arriva a influenzare praticamente tutti i vertici della rete. Tale andamento suggerisce l'esistenza, per WTSS, di una sorta di soglia critica di budget: al di sotto di essa il seed set prodotto non riesce ad avviare una diffusione globale, mentre una disponibilità sufficientemente elevata di risorse consente all'algoritmo di individuare una configurazione capace di innescare una cascata su scala quasi completa.

Tra gli altri metodi, My-Seeds mantiene prestazioni particolarmente competitive per gran parte dell'intervallo di budget considerato e, prima del salto finale di WTSS, rappresenta la strategia che ottiene la diffusione maggiore. Questo risultato è coerente con la logica dell'euristica proposta, che utilizza la coreness dei nodi e penalizza progressivamente candidati appartenenti a vicinati già coperti, cercando di distribuire il seed set in regioni strutturalmente rilevanti della rete evitando eccessive sovrapposizioni.

Il confronto tra le tre varianti di Cost-Seeds-Greedy evidenzia inoltre una netta superiorità di Greedy- f_3 rispetto a Greedy- f_1 e Greedy- f_2 in questo scenario basato sul grado. Greedy- f_3 mantiene una crescita più consistente e rimane competitivo con le euristiche WTSS e My-Seeds.

Nel complesso, questo secondo scenario mostra come la funzione di costo possa modificare profondamente il comportamento degli algoritmi. Sebbene c_1 e c_2 siano state costruite in modo da avere un costo medio e un costo complessivo della rete confrontabili, la diversa distribuzione dei costi produce dinamiche molto differenti. Quando il costo è indipendente dalla struttura, gli algoritmi possono selezionare anche nodi molto connessi senza subire una penalizzazione sistematica; quando invece il costo cresce con il grado, i nodi potenzialmente più influenti diventano anche più onerosi e il problema di selezione del seed set risulta sensibilmente più difficile.

Anche alla luce di questo secondo scenario, la scelta di $k^* = 10\%$ per i successivi esperimenti di robustezza appare adeguata. A tale livello di budget, nello scenario c_1 le soluzioni risultano già molto efficaci ma non tutte completamente sature, mentre nello scenario c_2

permane una chiara separazione tra i cinque algoritmi. Il valore selezionato rappresenta quindi un compromesso utile per valutare la robustezza delle soluzioni in condizioni sufficientemente significative per entrambe le funzioni di costo.

3 Esperimento 2 - Robustezza alla rimozione di archi

Il secondo esperimento è dedicato alla valutazione della robustezza dei seed set individuati dagli algoritmi rispetto a perturbazioni della struttura della rete. In particolare, si analizza come varia la diffusione quando una quota crescente degli archi del grafo viene rimossa casualmente, mantenendo invariato il seed set calcolato sul grafo originale.

Per questo esperimento il budget è stato fissato a un unico valore rappresentativo, indicato con k^* , pari al 10% del costo complessivo della rete. La scelta deriva dai risultati ottenuti nell'Esperimento 1. Nello scenario associato alla funzione di costo c_1 , tale livello di budget consente già di ottenere cascade molto estese senza trovarsi ancora, per tutti gli algoritmi, in una condizione di completa saturazione. Nello scenario c_2 , invece, il 10% mantiene ancora una chiara separazione tra le diverse strategie. Il valore selezionato costituisce pertanto un compromesso tra un regime di budget troppo restrittivo e il valore massimo del 20%, nel quale alcune configurazioni raggiungono già la quasi totalità della rete.

Per ogni funzione di costo e per ciascun algoritmo viene quindi recuperato il seed set S prodotto nell'Esperimento 1 con budget k^* . Tale insieme non viene ricalcolato dopo la perturbazione del grafo. L'obiettivo non è infatti determinare una nuova soluzione ottimizzata sulla rete modificata, ma verificare quanto una soluzione costruita conoscendo la struttura originale continui a essere efficace quando tale struttura subisce una perdita di collegamenti.

Indicando con

$$G = (V, E)$$

il grafo originale e con R_E l'insieme degli archi rimossi, il grafo perturbato è definito come

$$G' = (V, E \setminus R_E).$$

Su tale grafo viene nuovamente eseguito il processo di Majority Cascade utilizzando lo stesso seed set S , misurando il numero finale di nodi influenzati,

$$|Inf[G', S]|.$$

Sono stati considerati quattro livelli di perturbazione, corrispondenti alla rimozione dell'1%, 5%, 10% e 20% degli archi presenti nel grafo originale.

Un aspetto importante della procedura sperimentale riguarda il modo in cui sono stati costruiti gli insiemi di archi rimossi. **Le perturbazioni non sono state generate come quattro campioni casuali indipendenti.** È stato invece prodotto un unico ordinamento casuale degli archi e le rimozioni sono state effettuate in maniera progressiva, in modo che gli insiemi risultassero annidati:

$$R_{1\%} \subset R_{5\%} \subset R_{10\%} \subset R_{20\%}.$$

Di conseguenza, gli archi rimossi al livello dell'1% continuano a essere assenti anche negli scenari al 5%, 10% e 20%, ai quali vengono progressivamente aggiunti ulteriori archi. I diversi punti dell'esperimento rappresentano quindi livelli crescenti della stessa perturbazione casuale, anziché realizzazioni indipendenti tra loro.

Questa scelta risulta particolarmente utile poiché, per ciascun livello di perturbazione, è stata effettuata una **singola realizzazione casuale** e non una media su più ripetizioni.

Utilizzando perturbazioni cumulative si riduce il rischio che differenze tra due punti consecutivi siano dovute esclusivamente alla selezione di insiemi casuali completamente differenti.

L'andamento osservato può così essere interpretato più direttamente come conseguenza dell'aumento progressivo dell'intensità della stessa perturbazione.

La stessa logica verrà adottata anche nell'Esperimento 3, dedicato alla rimozione casuale di vertici. Anche in quel caso gli insiemi rimossi saranno costruiti in maniera annidata, in modo da avere

$$R_{1\%}^V \subset R_{5\%}^V \subset R_{10\%}^V \subset R_{20\%}^V.$$

In questo modo i due esperimenti di robustezza risultano coerenti dal punto di vista metodologico e permettono di confrontare l'effetto progressivo di due differenti tipi di perturbazione della rete.

Nel caso della rimozione degli archi **è stato inoltre imposto che nessun vertice diventasse completamente isolato**. Durante la costruzione della perturbazione, un arco viene quindi eliminato soltanto se la sua rimozione non porta il grado di uno dei due estremi a zero. Non viene invece richiesto che il grafo rimanga costituito da un'unica componente connessa: l'eventuale frammentazione della rete viene considerata come uno degli effetti della perturbazione e viene registrata durante l'analisi.

È infine importante osservare che la Majority Cascade viene simulata direttamente sul grafo perturbato G' . Di conseguenza, la soglia di attivazione di ciascun nodo viene determinata sulla base del nuovo grado,

$$t_{G'}(v) = \left\lceil \frac{d_{G'}(v)}{2} \right\rceil.$$

La rimozione degli archi può quindi produrre due effetti contrapposti: da un lato riduce il numero di collegamenti disponibili per la propagazione dell'influenza, dall'altro può diminuire la soglia necessaria affinché un nodo venga attivato.

La successiva analisi strutturale permette di quantificare entrambi gli effetti e costituisce la base per l'interpretazione dei risultati ottenuti.

3.1 Effetti strutturali della rimozione di archi

Prima di analizzare separatamente i risultati ottenuti con le due funzioni di costo, è utile osservare in che modo la rimozione progressiva degli archi modifica alcune caratteristiche strutturali della rete. Tale analisi è comune a entrambi gli scenari di costo, poiché per ciascun livello di perturbazione viene utilizzato lo stesso grafo modificato per tutti gli algoritmi.

Particolare attenzione è stata posta al grado medio e alla soglia media di attivazione. Nel modello Majority Cascade, infatti, la soglia di ciascun vertice dipende direttamente dal suo grado.

La rimozione di un arco può quindi avere un duplice effetto: da un lato riduce il numero di collegamenti attraverso i quali l'influenza può propagarsi, dall'altro può ridurre la soglia richiesta per l'attivazione di uno o di entrambi i vertici coinvolti.

La Tabella 2 riporta l'evoluzione delle principali grandezze strutturali considerate al crescere della percentuale di archi rimossi.

Tabella 2: Effetti strutturali della rimozione cumulativa di archi

Archi rimossi	Grado medio	Soglia media	Nodi con soglia ridotta	Componenti
0%	22.004	11.257	0%	1
1%	21.784	11.145	10.37%	2
5%	20.904	10.707	35.25%	5
10%	19.804	10.158	50.92%	8
20%	17.604	9.061	66.26%	22

I dati mostrano una progressiva diminuzione sia del grado medio sia della soglia media di attivazione. Già con la rimozione del 10% degli archi, oltre la metà dei vertici presenta una soglia inferiore rispetto a quella posseduta nel grafo originale; con una perturbazione del 20%, tale percentuale sale al 66.26%, mentre la soglia media passa da 11.257 a 9.061.

Parallelamente aumenta il numero di componenti connesse, che passa da una sola componente nel grafo originale a 22 nello scenario maggiormente perturbato. Nessun vertice risulta tuttavia isolato, in accordo con il vincolo imposto durante la procedura di rimozione. Questi risultati mostrano quindi come la perdita di connettività sia accompagnata da una progressiva riduzione delle condizioni necessarie all'attivazione dei nodi. Tale effetto sarà particolarmente importante nell'interpretazione delle curve di diffusione riportate nelle sottosezioni successive.

3.2 Funzione di costo c_1 (random)

La Figura 4 mostra l'andamento della diffusione ottenuta dai cinque algoritmi utilizzando la funzione di costo c_1 , al crescere della percentuale di archi rimossi. Il punto corri-

spondente allo 0% rappresenta la prestazione ottenuta sul grafo originale con budget fissato a $k^* = 10\%$, mentre i punti successivi riportano il numero di nodi influenzati dopo l'applicazione delle perturbazioni cumulative descritte in precedenza.

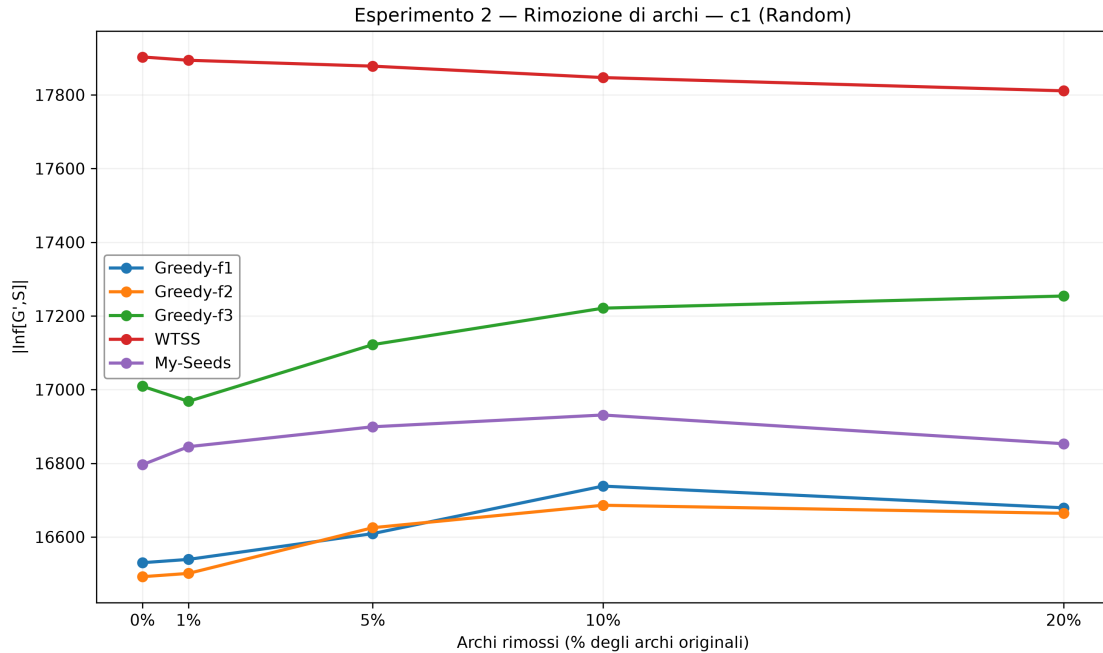


Figura 4: Numero di nodi influenzati al crescere della percentuale di archi rimossi per la funzione di costo c_1 .

L'elemento più evidente è la notevole stabilità delle soluzioni rispetto alla rimozione degli archi. Anche quando viene eliminato il 20% dei collegamenti originari, nessuno degli algoritmi mostra una riduzione marcata della dimensione della cascata. Le variazioni osservate rimangono infatti relativamente contenute e, per alcune strategie, la diffusione aumenta leggermente rispetto a quella misurata sul grafo originale.

- WTSS rappresenta il caso più immediato. Con il budget k^* l'algoritmo parte già da una situazione prossima alla saturazione della rete, influenzando quasi tutti i 17 903 vertici. La progressiva rimozione degli archi determina solamente una lieve diminuzione della diffusione, che rimane comunque superiore a 17 800 nodi anche nello scenario più severo. La ridotta variazione è quindi in parte riconducibile a un effetto di saturazione: essendo la cascata iniziale già molto vicina al massimo teoricamente raggiungibile, lo spazio disponibile per variazioni significative è limitato.
- Un comportamento differente si osserva per le tre varianti di Cost-Seeds-Greedy. Greedy- f_1 e Greedy- f_2 non mostrano una degradazione progressiva all'aumentare degli archi rimossi. Al contrario, la dimensione della cascata cresce leggermente fino alla perturbazione del 10% e diminuisce solo marginalmente al 20%, mantenendosi comunque su

valori comparabili, e in alcuni casi leggermente superiori, a quelli osservati sul grafo originale. Le due curve rimangono inoltre molto vicine tra loro lungo l'intero intervallo di perturbazione.

- Greedy- f_3 presenta un andamento ancora più interessante. Dopo una lieve riduzione in corrispondenza dell'1% di archi rimossi, la diffusione aumenta progressivamente al 5%, 10% e 20%, raggiungendo nello scenario maggiormente perturbato un numero di nodi influenzati superiore a quello ottenuto sul grafo originale. La perdita di collegamenti non determina quindi, in questo caso, un deterioramento della soluzione, ma è associata a un leggero incremento della cascata.
- Anche My-Seeds mostra una buona stabilità. La diffusione aumenta inizialmente con la perturbazione e raggiunge il valore massimo intorno al 10% di archi rimossi; soltanto al 20% si osserva una lieve diminuzione. Anche in questo caso, tuttavia, la variazione rispetto alla situazione iniziale rimane contenuta.

L'assenza di un decadimento netto della diffusione può essere interpretata alla luce dell'analisi strutturale riportata nella Tabella 2. La progressiva perdita di collegamenti è infatti accompagnata da una diminuzione delle soglie majority per una quota crescente dei vertici. La minore disponibilità di archi per la propagazione viene quindi parzialmente compensata da condizioni di attivazione meno restrittive.

Nel caso della funzione di costo c_1 , tale compensazione risulta particolarmente evidente: fino al 20% di archi rimossi, i cinque seed set mantengono una capacità di diffusione molto vicina a quella osservata sul grafo originale e, per alcune strategie, il numero di nodi influenzati **aumenta leggermente**. I risultati evidenziano quindi una notevole robustezza rispetto alla perdita casuale di collegamenti, mostrando al tempo stesso una caratteristica peculiare del Majority Cascade: una riduzione della connettività non comporta necessariamente una riduzione proporzionale della diffusione, poiché modifica contemporaneamente anche le soglie di attivazione.

3.3 Funzione di costo c_2 (grado)

La Figura 5 mostra l'andamento della diffusione ottenuta dai cinque algoritmi nello scenario basato sulla funzione di costo c_2 , mantenendo fisso il budget a $k^* = 10\%$ e applicando livelli crescenti di rimozione cumulativa degli archi.

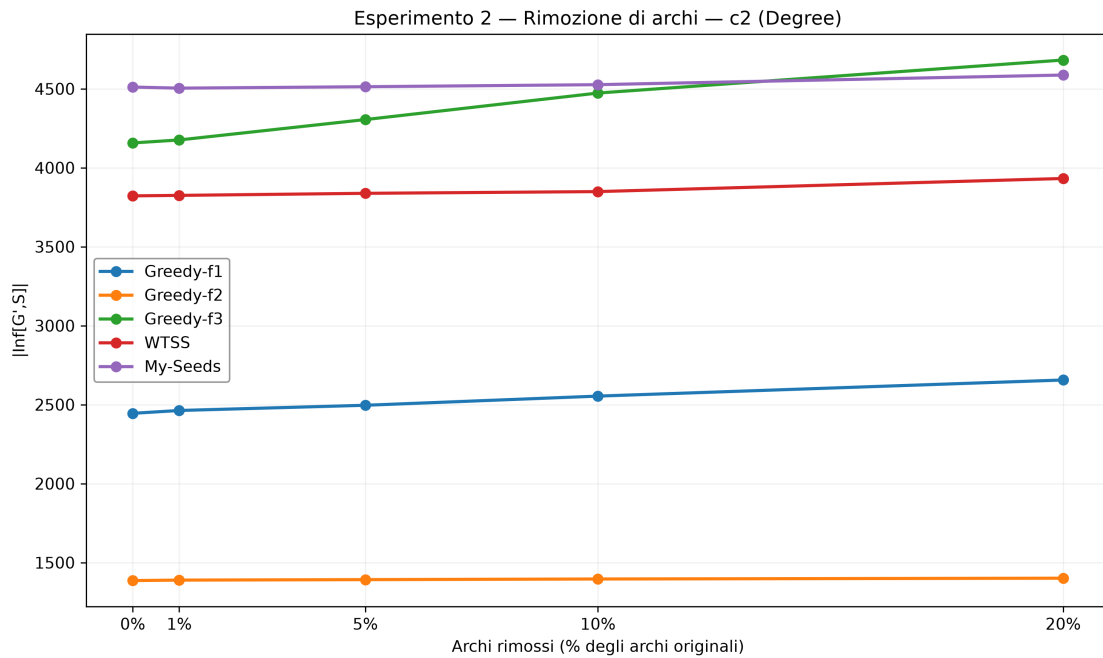


Figura 5: Numero di nodi influenzati al crescere della percentuale di archi rimossi per la funzione di costo c_2 .

Anche in questo secondo scenario la rimozione degli archi non determina un decadimento significativo della diffusione. Il risultato è particolarmente interessante perché, a differenza di quanto osservato con c_1 , le cascade ottenute con c_2 sono lontane dalla saturazione della rete. La sostanziale stabilità delle curve riflette maggiormente l'interazione tra perdita di collegamenti e riduzione delle soglie majority evidenziata nell'analisi strutturale precedente.

- My-Seeds presenta inizialmente la diffusione più elevata, con circa 4 500 nodi influenzati sul grafo originale. Il valore rimane pressoché invariato nei primi livelli di perturbazione e cresce leggermente al 10% e al 20% di archi rimossi. L'algoritmo mantiene quindi una notevole stabilità rispetto alle modifiche della struttura della rete.
- Greedy- f_3 mostra invece la crescita più evidente. Partendo da una diffusione inferiore rispetto a My-Seeds, il numero di nodi influenzati aumenta progressivamente all'intensificarsi della perturbazione. Al 20% Greedy- f_3 supera My-Seeds e diventa il metodo con la maggiore diffusione nello scenario considerato. Anche in questo caso, quindi, la rimozione degli archi non compromette il seed set originariamente selezionato ma, al contrario, può favorire l'estensione della cascata.
- WTSS presenta un comportamento analogo ma più contenuto: la diffusione rimane quasi costante per i primi livelli di rimozione e cresce moderatamente con le perturbazioni più elevate. Greedy- f_1 mostra anch'esso un incremento graduale, passando da circa 2 400 nodi influenzati sul grafo originale a oltre 2 600 nello scenario al 20%. Greedy- f_2 ,

infine, rimane sostanzialmente invariato lungo tutto l'intervallo, mantenendo tuttavia il livello di diffusione più basso tra i metodi confrontati.

Nel complesso, le differenze relative tra gli algoritmi restano quindi abbastanza stabili, ma nessuna delle strategie evidenzia una perdita significativa di efficacia. Al contrario, quattro dei cinque metodi mostrano un leggero incremento del numero di nodi influenzati all'aumentare degli archi rimossi.

Tale comportamento è coerente con i risultati riportati nella Tabella 2. Con l'aumentare della perturbazione, una quota crescente dei vertici presenta una soglia di attivazione inferiore rispetto al grafo originale: al 10% di archi rimossi ciò riguarda oltre la metà dei nodi, mentre al 20% la percentuale supera il 66%.

Nel caso di c_2 , dove le cascade iniziali sono molto più lontane dalla saturazione rispetto allo scenario random, l'effetto della riduzione delle soglie risulta particolarmente visibile e può tradursi direttamente in un aumento di $|Inf[G', S]|$.

I risultati confermano quindi che la robustezza osservata nell'Esperimento 2 non dipende esclusivamente dalla funzione di costo utilizzata. Pur producendo seed set e livelli di diffusione molto diversi, sia c_1 sia c_2 mostrano una forte resistenza alla perdita casuale di collegamenti. La caratteristica comune è legata al funzionamento stesso del Majority Cascade: la progressiva riduzione del grado dei nodi può rendere più semplice il raggiungimento della maggioranza richiesta per l'attivazione, compensando almeno in parte il deterioramento strutturale della rete.

4 Esperimento 3 - Robustezza alla rimozione di vertici

Il terzo esperimento analizza la robustezza dei seed set rispetto a una perturbazione più invasiva della struttura della rete, ottenuta attraverso la rimozione casuale di una quota crescente dei vertici. Analogamente all'Esperimento 2, il budget viene mantenuto fisso al valore rappresentativo $k^* = 10\%$ e, per ciascuna funzione di costo e ciascun algoritmo, viene riutilizzato il seed set calcolato sull'istanza originale nell'Esperimento 1.

Indicando con

$$G = (V, E)$$

il grafo originale e con $R_V \subseteq V$ l'insieme dei vertici rimossi, il grafo perturbato è definito come il sottografo indotto

$$G' = G[V \setminus R_V].$$

A differenza della rimozione degli archi, questa perturbazione modifica direttamente anche l'insieme dei nodi disponibili. Un vertice appartenente al seed set originale può infatti essere incluso nell'insieme R_V e risultare quindi assente dal grafo perturbato. Per questo motivo, il seed set effettivamente utilizzato nella simulazione diventa

$$S' = S \setminus R_V.$$

I seed rimossi non vengono sostituiti con nuovi vertici. Una loro eventuale sostituzione richiederebbe infatti una nuova esecuzione dell'algoritmo di selezione sul grafo perturbato e non permetterebbe più di valutare la robustezza della soluzione originariamente costruita. L'**obiettivo** dell'esperimento consiste invece nel verificare quanto il seed set ottenuto conoscendo la rete originale continui a essere efficace quando una parte dei vertici, eventualmente comprendente alcuni seed, non è più disponibile.

Come nell'Esperimento 2, sono stati considerati quattro livelli di perturbazione, corrispondenti alla rimozione dell'1%, 5%, 10% e 20% dei vertici del grafo originale. Anche in questo caso gli **insiemi rimossi sono stati costruiti in maniera cumulativa**. A partire da un unico ordinamento casuale dei vertici, per ciascun livello di perturbazione è stato utilizzato un prefisso crescente della stessa sequenza, ottenendo

$$R_{1\%}^V \subset R_{5\%}^V \subset R_{10\%}^V \subset R_{20\%}^V.$$

Pertanto, tutti i nodi eliminati nello scenario all'1% continuano a essere assenti negli scenari successivi, ai quali vengono progressivamente aggiunti ulteriori vertici. Come per la rimozione degli archi, tale scelta permette di interpretare i quattro livelli come un progressivo aggravamento della stessa perturbazione casuale e limita la variabilità che deriverebbe dall'utilizzo di campioni indipendenti, particolarmente rilevante poiché per ciascuna percentuale viene considerata una singola realizzazione e non una media su più esecuzioni.

Per ogni livello di perturbazione viene utilizzato lo stesso insieme di vertici rimossi per tutti gli algoritmi e per entrambe le funzioni di costo. In questo modo le differenze osservate tra i metodi non possono essere attribuite all'utilizzo di istanze perturbate differenti, ma dipendono esclusivamente dalle caratteristiche dei seed set prodotti.

A differenza di quanto fatto nell'Esperimento 2 per i nodi isolati, nella rimozione dei vertici **non viene imposto alcun vincolo volto a preservare la connettività del grafo**. L'eventuale frammentazione della rete costituisce infatti uno degli effetti naturali della perturbazione che si intende osservare. Per questo motivo vengono registrati, per ciascun livello di rimozione, il numero di componenti connesse e la dimensione della componente connessa più grande.

La simulazione del Majority Cascade viene quindi eseguita sul grafo perturbato G' utilizzando il seed set sopravvissuto S' . La quantità principale osservata rimane

$$|Inf[G', S']|,$$

ossia il numero complessivo di vertici attivati al termine del processo.

Anche in questo esperimento le soglie majority vengono calcolate rispetto alla struttura del grafo perturbato.

La rimozione dei vertici, oltre a modificare i gradi e le soglie dei nodi sopravvissuti, riduce direttamente la dimensione della rete, elimina tutti gli archi incidenti ai vertici rimossi, può aumentare la frammentazione del grafo e può colpire direttamente alcuni nodi appartenenti al seed set. Per questo motivo, accanto alla dimensione assoluta della cascata, verranno considerate anche informazioni relative alla struttura residua della rete e alla quota di seed che rimane disponibile dopo ciascuna perturbazione.

La successiva analisi strutturale permette di quantificare tali effetti prima di esaminare separatamente i risultati ottenuti con le due funzioni di costo.

4.1 Effetti strutturali della rimozione di vertici

Come nell'Esperimento 2, tale analisi è indipendente dalla funzione di costo utilizzata, poiché per ogni livello di perturbazione lo stesso grafo modificato viene impiegato per tutti gli algoritmi.

La rimozione di un vertice produce un effetto più incisivo rispetto alla rimozione di un singolo arco, poiché determina contemporaneamente la scomparsa del nodo e di tutti gli archi ad esso incidenti. Ne consegue una riduzione sia della dimensione della rete sia del grado dei vertici sopravvissuti. Poiché la soglia di attivazione nel Majority Cascade è dipendente dal grado del nodo, anche in questo caso la perturbazione può ridurre le soglie richieste per l'attivazione. A differenza dell'Esperimento 2, tuttavia, tale possibile facilitazione della cascata è accompagnata dalla perdita diretta di una quota dei nodi della rete e, potenzialmente, di alcuni vertici appartenenti al seed set.

La Tabella 3 riassume le principali variazioni strutturali osservate al crescere della percentuale di vertici rimossi.

Tabella 3: Effetti strutturali della rimozione cumulativa di vertici

Vertici rimossi	Vertici rimanenti	Grado medio	Soglia media	Nodi con soglia ridotta	Componenti	LCC sui nodi rimanenti
0%	17 903	22.004	11.257	0%	1	100.000%
1%	17 724	21.832	11.169	9.50%	12	99.927%
5%	17 008	20.931	10.717	35.21%	72	99.283%
10%	16 113	19.751	10.127	52.02%	158	98.225%
20%	14 322	17.538	9.023	68.36%	311	96.551%

Come prevedibile, l'aumento della perturbazione determina una progressiva diminuzione del grado medio, che passa da 22.004 nel grafo originale a 17.538 dopo la rimozione del 20% dei vertici. Parallelamente, la soglia media di attivazione scende da 11.257 a 9.023. Già al livello del 10%, più della metà dei nodi sopravvissuti presenta una soglia inferiore rispetto a quella posseduta nella rete originale; al 20% tale quota raggiunge il 68.36%.

La perturbazione produce inoltre una progressiva frammentazione della rete. Il numero di componenti connesse cresce da 1 a 311 nello scenario più severo. Nonostante ciò, la componente connessa più grande continua a contenere una quota molto elevata dei vertici

sopravvissuti: anche dopo la rimozione del 20% dei nodi, essa comprende circa il 96.55% della rete residua. La frammentazione è quindi principalmente dovuta alla formazione di numerose componenti di dimensioni ridotte, mentre il nucleo principale della rete rimane largamente preservato.

Questi risultati evidenziano un'importante differenza rispetto alla rimozione degli archi. Anche in questo caso la diminuzione dei gradi rende meno restrittive le soglie majority, ma tale effetto favorevole è accompagnato dalla riduzione diretta del numero massimo di nodi che possono essere influenzati e dalla possibile eliminazione di alcuni elementi del seed set. Di conseguenza, una diminuzione di $|Inf[G', S']|$ non può essere interpretata automaticamente come una perdita proporzionale di efficacia dell'algoritmo: una parte della riduzione deriva semplicemente dal fatto che il grafo perturbato contiene meno vertici.

Per questo motivo, nelle sottosezioni successive il numero assoluto di nodi influenzati sarà interpretato anche tenendo conto della dimensione della rete residua e della quota di seed che sopravvive alla perturbazione.

4.2 Funzione di costo c_1 (random)

La Figura 6 mostra l'andamento della diffusione ottenuta con la funzione di costo c_1 al crescere della percentuale di vertici rimossi. Il punto corrispondente allo 0% rappresenta la prestazione sul grafo originale con budget $k^* = 10\%$, mentre i punti successivi fanno riferimento alle perturbazioni cumulative definite in precedenza.

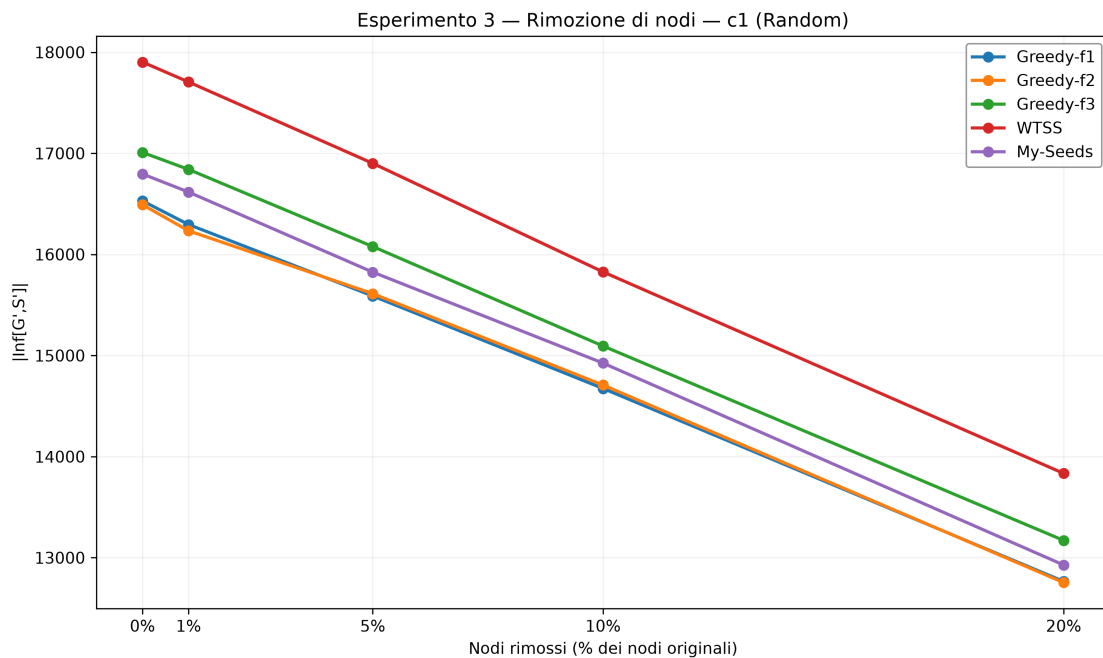


Figura 6: Numero di nodi influenzati al crescere della percentuale di vertici rimossi per la funzione di costo c_1 .

A differenza di quanto osservato nell'Esperimento 2, la rimozione dei vertici determina una diminuzione chiara e progressiva del numero assoluto di nodi influenzati per tutti gli algoritmi. Le curve mantengono tuttavia un **andamento abbastanza regolare** e le posizioni relative dei diversi metodi rimangono sostanzialmente invariate lungo l'intero intervallo di perturbazione.

- WTSS conserva la diffusione più elevata a ogni livello di rimozione. Partendo da una situazione prossima alla saturazione sul grafo originale, la dimensione della cascata diminuisce progressivamente, ma anche nello scenario più severo rimane superiore a quella ottenuta dagli altri metodi. Greedy- f_3 mantiene la seconda prestazione migliore, seguito da My-Seeds, mentre Greedy- f_1 e Greedy- f_2 presentano valori molto simili tra loro.

La diminuzione osservata nel grafico deve tuttavia essere interpretata tenendo conto del fatto che la dimensione stessa della rete si riduce. Al 20% di perturbazione, infatti, il grafo contiene 14 322 vertici invece dei 17 903 originari. Una parte rilevante della diminuzione di $|Inf[G', S']|$ è quindi inevitabile, poiché i nodi rimossi non possono più essere raggiunti dalla cascata.

Questo aspetto emerge chiaramente confrontando il numero assoluto di nodi influenzati con la quota della rete residua effettivamente raggiunta. Anche dopo la rimozione del 20% dei vertici, Greedy- f_1 e Greedy- f_2 influenzano circa l'89% dei nodi ancora presenti, My-Seeds circa il 90.3% e Greedy- f_3 circa il 92.0%. WTSS conserva invece una copertura particolarmente elevata, pari a circa il 96.6% della rete residua.

La robustezza risulta evidente anche considerando la diffusione conservata rispetto al grafo originale. Nello scenario al 10% di rimozione, tutti i metodi mantengono circa l'88–89% della propria diffusione iniziale; al 20%, tale valore rimane prossimo al 77%. **La perdita assoluta osservata nel grafico risulta quindi sostanzialmente proporzionata alla riduzione della dimensione della rete.**

Un ulteriore effetto della perturbazione riguarda direttamente il seed set. Poiché i nodi vengono eliminati casualmente e i seed rimossi non vengono sostituiti, la dimensione del seed set disponibile diminuisce progressivamente. I risultati mostrano tuttavia un comportamento molto regolare: con il 10% dei vertici rimossi sopravvive circa il 90% dei seed originari, mentre al 20% ne rimane circa l'80%. Non emergono quindi evidenze che una particolare strategia venga colpita in maniera anomala dalla rimozione casuale; la perdita dei seed segue approssimativamente la stessa intensità della perturbazione applicata alla rete.

Nel complesso, lo scenario c_1 mostra quindi una buona robustezza rispetto alla rimozione casuale di vertici. La dimensione assoluta della cascata diminuisce in maniera evidente, ma tale andamento è in larga parte riconducibile alla riduzione del numero di nodi disponibili e alla perdita proporzionale di una parte del seed set. Considerando invece la rete residua, tutti gli algoritmi continuano a raggiungerne una quota molto elevata, con WTSS che mantiene la maggiore capacità di diffusione anche nelle condizioni di perturbazione più severe.

4.3 Funzione di costo c_2 (grado)

La Figura 7 mostra l'andamento della diffusione ottenuta nello scenario basato sulla funzione di costo c_2 , mantenendo fisso il budget a $k^* = 10\%$ e applicando livelli crescenti di rimozione cumulativa dei vertici.

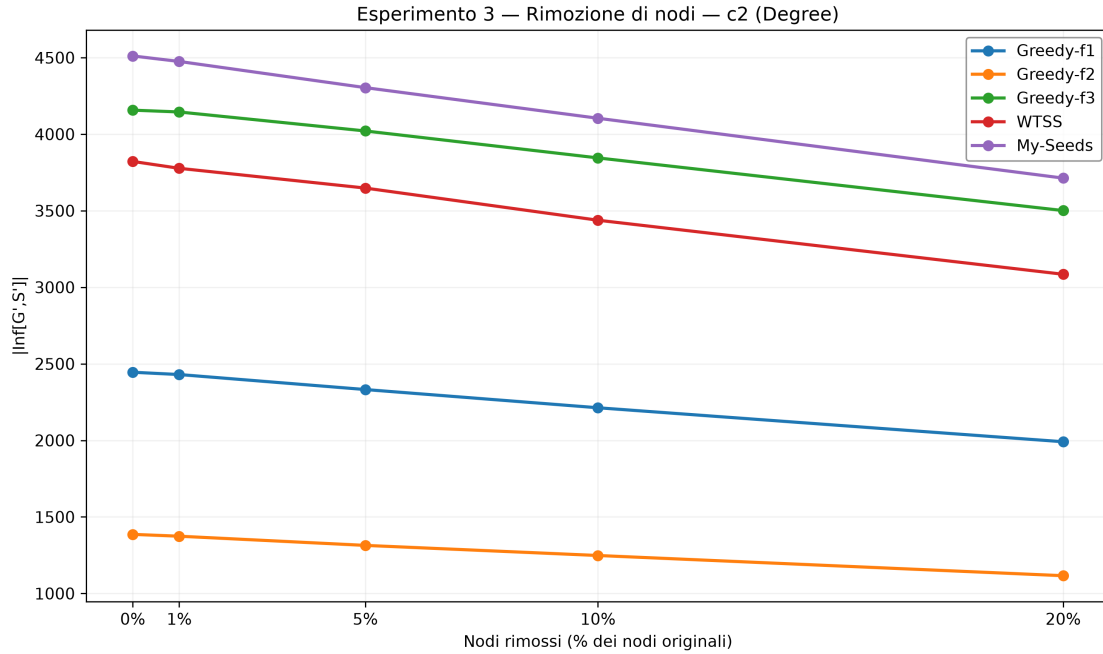


Figura 7: Numero di nodi influenzati al crescere della percentuale di vertici rimossi per la funzione di costo c_2 .

Anche in questo scenario tutte le curve mostrano una diminuzione progressiva del numero assoluto di nodi influenzati. Rispetto a quanto osservato con c_1 , tuttavia, le cascade partono da valori molto più contenuti e rimangono lontane dalla saturazione della rete per tutti i livelli di perturbazione.

- My-Seeds mantiene la diffusione più elevata lungo l'intero intervallo considerato. Sul grafo originale influenza circa 4 500 nodi e, pur diminuendo progressivamente al crescere della perturbazione, conserva il vantaggio sugli altri metodi anche dopo la rimozione del 20% dei vertici. Greedy- f_3 occupa stabilmente la seconda posizione, seguito da WTSS, Greedy- f_1 e Greedy- f_2 .

L'ordinamento degli algoritmi rimane quindi sostanzialmente invariato rispetto al caso non perturbato. Questo indica che la rimozione casuale dei vertici non modifica in maniera significativa i rapporti di efficacia tra le diverse strategie, ma determina principalmente una riduzione progressiva della diffusione ottenuta da ciascuna di esse.

Osservando la diffusione conservata rispetto al grafo originale, emerge inoltre una **buona stabilità relativa delle soluzioni**. Con una rimozione del 10% dei vertici, Greedy- f_1 ,

Greedy- f_2 , My-Seeds e WTSS mantengono circa il 90–91% della diffusione iniziale, mentre Greedy- f_3 raggiunge circa il 92.5%. Al livello massimo di perturbazione, pari al 20%, la diffusione conservata rimane compresa tra circa l'80.5% e l'84.2%, con Greedy- f_3 che presenta il valore più elevato.

Anche la perdita dei seed segue un andamento molto regolare. Al 10% di vertici rimossi sopravvive approssimativamente il 90% del seed set originario per tutti gli algoritmi, mentre al 20% la quota residua si colloca intorno all'80%. My-Seeds conserva una percentuale leggermente superiore, pari a circa l'81.1%, mentre WTSS risulta leggermente più penalizzato, con circa il 79.1% dei seed ancora disponibili. Le differenze sono comunque contenute e coerenti con la natura casuale della perturbazione.

Nel complesso, anche nello scenario c_2 le soluzioni mostrano una buona robustezza rispetto alla rimozione casuale dei nodi. My-Seeds conserva la migliore capacità di diffusione assoluta, mentre Greedy- f_3 evidenzia la maggiore stabilità relativa rispetto alla propria prestazione iniziale. Più in generale, la perturbazione riduce progressivamente la dimensione delle cascade senza modificare sostanzialmente le differenze tra gli algoritmi, confermando che le caratteristiche dei seed set individuati sul grafo originale rimangono efficaci anche in presenza di una perdita significativa di vertici.

5 Analisi comparativa e conclusioni

5.1 Confronto sull'effetto del budget e della funzione di costo

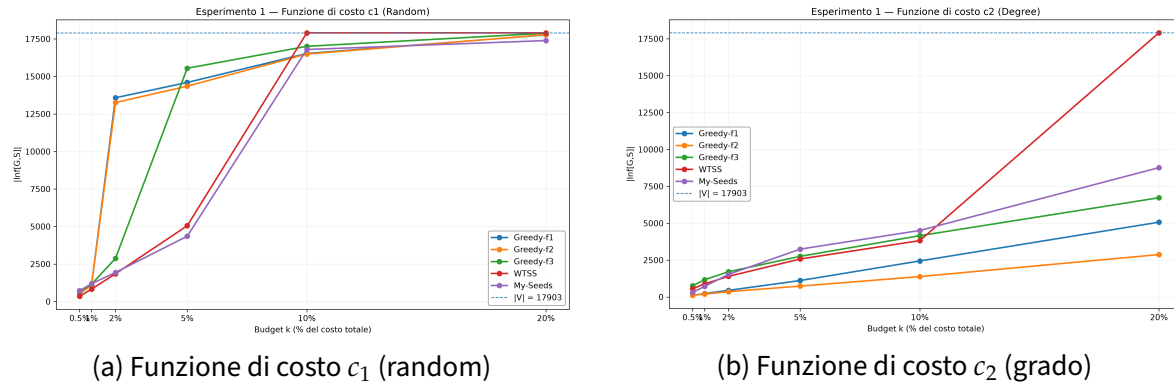


Figura 8: Confronto dei risultati dell'Esperimento 1 per le due funzioni di costo.

Il primo esperimento mostra chiaramente come il comportamento degli algoritmi dipenda sia dal budget disponibile sia dalla funzione di costo adottata. Nel caso della funzione c_1 , i **costi sono indipendenti dalla struttura della rete** e questo consente agli algoritmi di selezionare anche nodi molto connessi senza penalizzazioni sistematiche. Di conseguenza, la diffusione cresce rapidamente all'aumentare del budget e diversi metodi raggiungono condizioni prossime alla saturazione già per valori intermedi di k .

Lo scenario cambia in modo evidente utilizzando $c_2(v) = \lceil d(v)/2 \rceil$. In questo caso, i nodi ad alto grado risultano anche più costosi e diventano quindi più difficili da includere nel seed set. Le cascade risultano generalmente meno estese e le differenze tra gli algoritmi rimangono visibili anche per budget relativamente elevati.

Dal confronto emerge inoltre che **non esiste un algoritmo dominante per tutti i livelli di budget**. Greedy- f_1 e Greedy- f_2 risultano particolarmente efficaci nello scenario c_1 con budget ridotti, mentre Greedy- f_3 assume un ruolo più competitivo nello scenario basato sul grado. My-Seeds ottiene risultati particolarmente interessanti con c_2 per budget intermedi, mentre WTSS mostra una crescita più marcata quando il budget diventa sufficientemente elevato.

Nel complesso, l'Esperimento 1 evidenzia quindi che **la funzione di costo modifica profondamente sia la dimensione della diffusione sia la posizione relativa degli algoritmi**. La scelta di $k^* = 10\%$ per i successivi esperimenti di robustezza deriva proprio da questo confronto: tale valore evita sia il regime fortemente restrittivo dei budget più bassi sia le condizioni di saturazione osservate, soprattutto con c_1 , al livello massimo del 20%.

5.2 Confronto sulla robustezza alla rimozione di archi

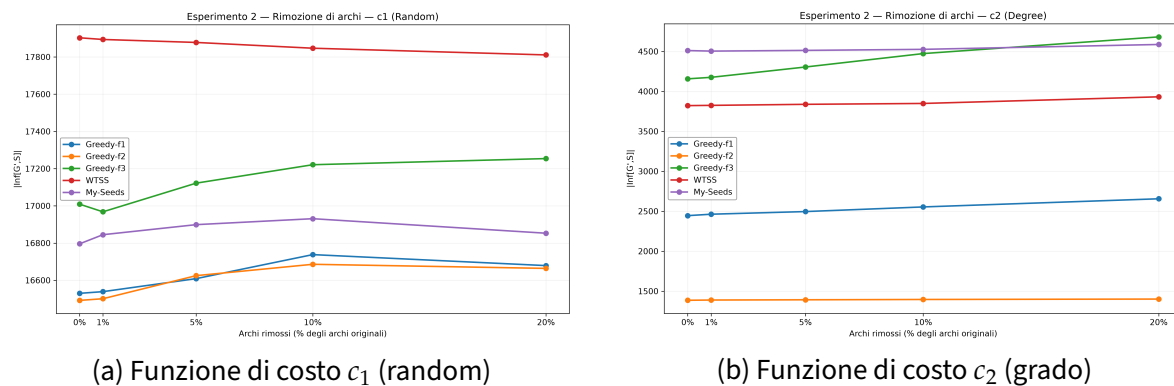


Figura 9: Confronto della robustezza alla rimozione di archi per le due funzioni di costo.

Il secondo esperimento produce un risultato particolarmente significativo: **la rimozione casuale di archi non determina una riduzione rilevante della diffusione**. Per entrambe le funzioni di costo, le curve rimangono infatti relativamente stabili anche quando viene eliminato il 20% degli archi originali; in alcuni casi il numero di nodi influenzati aumenta leggermente.

Nel caso di c_1 , parte di questa stabilità può essere ricondotta al fatto che diversi algoritmi partono già da una diffusione prossima alla saturazione della rete. Tuttavia, lo stesso comportamento emerge anche con c_2 , dove le cascade iniziali sono molto più contenute. Ciò indica che il fenomeno non può essere spiegato esclusivamente attraverso un effetto soffitto (i valori si concentrano verso il limite massimo).

L'analisi strutturale permette di individuare una spiegazione più generale. La rimozione degli archi riduce il grado dei vertici e, di conseguenza, anche le soglie majority. Al 20% di rimozione, il 66.26% dei nodi presenta una soglia inferiore rispetto al grafo originale. Pertanto, alla perdita di connettività si accompagna una condizione di attivazione meno restrittiva.

Il risultato principale dell'esperimento è quindi che **la perdita di archi e la riduzione delle soglie tendono a compensarsi**. Nel Majority Cascade, una rete strutturalmente più povera di collegamenti non implica necessariamente una diffusione inferiore, poiché gli stessi nodi possono richiedere un numero minore di vicini attivi per entrare nella cascata.

5.3 Confronto sulla robustezza alla rimozione di vertici

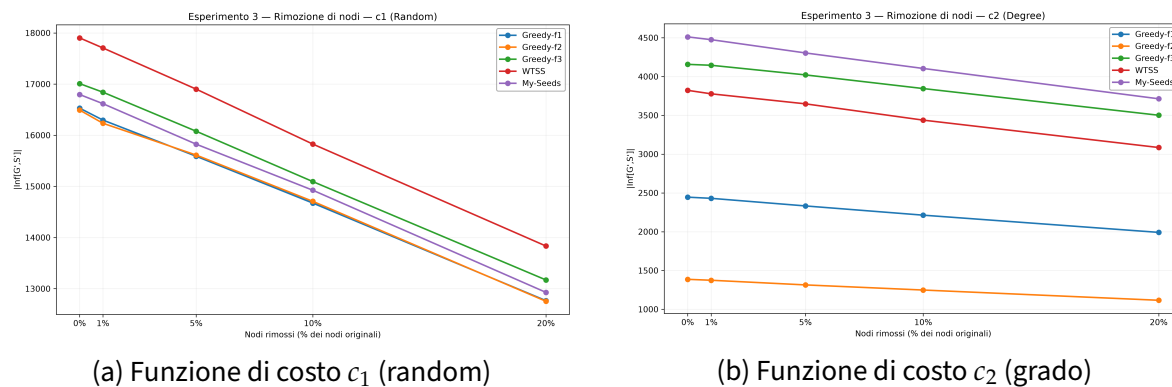


Figura 10: Confronto della robustezza alla rimozione di vertici per le due funzioni di costo.

La rimozione dei vertici produce un comportamento differente rispetto alla perturbazione degli archi. In questo caso, tutte le curve mostrano una diminuzione progressiva del numero assoluto di nodi influenzati all'aumentare della percentuale di vertici rimossi.

La riduzione è tuttavia in larga parte conseguenza della diminuzione della dimensione stessa della rete. Con una perturbazione del 20%, il numero di vertici passa infatti da 17 903 a 14 322. A questo si aggiunge la possibilità che alcuni nodi appartenenti al seed set vengano eliminati direttamente, riducendo quindi anche la dimensione di S' .

Nonostante ciò, i risultati mostrano che **i seed set mantengono una buona efficacia sulla rete residua**. Con c_1 , WTSS continua a raggiungere la quota più elevata dei nodi sopravvissuti, mentre Greedy- f_3 e My-Seeds mantengono prestazioni molto competitive. Con c_2 , My-Seeds conserva la maggiore diffusione assoluta, mentre Greedy- f_3 mostra una stabilità relativa particolarmente elevata rispetto alla propria prestazione iniziale.

Anche la struttura della rete rimane sorprendentemente resistente. Sebbene il numero di componenti connesse aumenti con la perturbazione, al 20% la componente connessa più grande contiene ancora oltre il 96% dei vertici sopravvissuti.

Si può quindi concludere che **la rimozione di vertici ha un impatto più evidente della rimozione di archi**, ma non provoca un collasso della capacità di diffusione. Una quota consistente della diminuzione osservata dipende semplicemente dalla perdita fisica dei nodi e dei seed, mentre gli algoritmi continuano a essere efficaci sulla porzione di rete che rimane disponibile.

5.4 Sintesi e considerazioni conclusive

Tabella 4: Sintesi dei principali risultati sperimentali

Aspetto	Risultato principale
Effetto del budget	L'aumento di k incrementa la diffusione, ma oltre determinati livelli emergono fenomeni di rendimento decrescente e saturazione.
Funzione di costo c_1	Produce generalmente cascade più estese, poiché il costo dei nodi è indipendente dalla loro posizione strutturale nella rete.
Funzione di costo c_2	Rende la selezione più complessa, poiché i nodi ad alto grado risultano anche più costosi.
Rimozione di archi	La diffusione rimane molto stabile; la riduzione delle soglie majority compensa in parte la perdita di connettività.
Rimozione di vertici	Determina una riduzione più evidente della cascata, dovuta alla diminuzione della rete, alla perdita di archi incidenti e alla possibile eliminazione di seed.
Confronto tra algoritmi	Non emerge un vincitore assoluto: le prestazioni dipendono dal budget, dalla funzione di costo e dal tipo di perturbazione considerato.

Considerando congiuntamente i tre esperimenti, emerge innanzitutto che **la funzione di costo rappresenta uno degli elementi più determinanti del problema**. Quando il costo è indipendente dalla struttura del grafo, come nel caso di c_1 , gli algoritmi riescono generalmente a costruire seed set capaci di generare cascade molto estese. Quando invece il costo cresce con il grado, come in c_2 , i nodi maggiormente connessi diventano più onerosi e la diffusione risulta sensibilmente più difficile.

Anche il budget svolge un ruolo fondamentale. L'aumento di k produce inizialmente incrementi rilevanti della diffusione, ma per valori elevati emergono fenomeni di rendimento decrescente e, in alcuni scenari, di vera e propria saturazione. Questo conferma che un aumento delle risorse disponibili non produce necessariamente un beneficio proporzionale.

Dal punto di vista della robustezza, la differenza più significativa riguarda il tipo di perturbazione applicata. **La rimozione di archi risulta molto meno dannosa della rimozione**

di vertici: nel primo caso la diminuzione del grado riduce contemporaneamente le soglie majority e può compensare la perdita di connettività; nel secondo caso vengono invece rimossi direttamente nodi, archi incidenti ed eventualmente elementi del seed set.

Infine, il confronto tra gli algoritmi non individua un vincitore assoluto. Greedy- f_1 e Greedy- f_2 risultano molto efficaci in alcuni scenari con costo random, Greedy- f_3 mostra un comportamento più equilibrato e competitivo soprattutto con c_2 , WTSS raggiunge prestazioni particolarmente elevate quando il budget è sufficientemente ampio, mentre My-Seeds si dimostra competitivo soprattutto nello scenario in cui il costo dipende dal grado.

Nel complesso, i risultati mostrano quindi che l'efficacia di una strategia di seed selection non può essere valutata indipendentemente dal budget, dalla funzione di costo e dalle caratteristiche strutturali della rete. Allo stesso tempo, la rete ca-AstroPh e il processo di Majority Cascade evidenziano una notevole capacità di mantenere la diffusione anche in presenza di perturbazioni strutturali significative.